

提示 4-10: 编写函数时, 应尽量保证该函数能对任何合法参数得到正确的结果。如若不然, 应在显著位置标明函数的缺陷, 以避免误用。

下面是改进之后的版本:

程序 4-4 素数判定 (2)

```
int is_prime(int n)
{
    if(n <= 1) return 0;
    int m = floor(sqrt(n) + 0.5);
    for(int i = 2; i <= m; i++)
        if(n % i == 0) return 0;
    return 1;
}
```

除了特判 $n \leq 1$ 的情况外, 程序中还使用了变量 m , 一方面避免了每次重复计算 $\text{sqrt}(n)$, 另一方面也通过四舍五入避免了浮点误差——正如前面所说, 如果 sqrt 将某个本应是整数的值变成了 xxx.99999 , 也将被修正, 但若直接写 $m = \text{sqrt}(n)$, “.99999” 会被直接截掉。

为什么 is_prime 的参数不是 long long 型呢? 因为当 n 很大时, 上述函数并不能很快计算出结果。对此, 在竞赛篇会有更详细的讨论。

4.2 函数调用与参数传递

4.1 节介绍的数学函数的特点是: 做计算, 然后返回一个值。但有时要做的并不是“计算”——如交换两个变量; 而有时则需要返回两个甚至更多的值——如解一个二元一次方程组, 函数仍然能满足需求, 但是规则会更复杂。根据笔者的经验, 这部分知识没搞清楚的初学者很容易在实战时出错, 所以这里介绍一些原理性的知识, 虽然有些枯燥, 但能帮助读者更好地理解。

4.2.1 形参与实参

程序 4-5 用函数交换变量 (错误)

```
#include<stdio.h>
void swap(int a, int b)
{
    int t = a; a = b; b = t;
}

int main()
{
    int a = 3, b = 4;
```

```

swap(3, 4);
printf("%d %d\n", a, b);
return 0;
}

```

读者应当还记得，这就是三变量交换算法。下面测试一下这个函数是否好用。很不幸，输出是“3 4”，而不是“4 3”。事实上，*a* 和 *b* 并没有被交换。为什么会这样呢？为了解释这一问题，请回忆“赋值”这个重要概念的含义。“诡异”的赋值语句 *a* = *a* + 1 是这样解释的：分为两步，首先计算赋值符号右边的 *a* + 1，然后把它装入变量 *a*，覆盖原来的值。那函数调用的过程又是怎样的呢？

第 1 步，计算参数的值。在上面的例子中，因为 *a* = 3, *b* = 4，所以 *swap*(*a*, *b*) 等价于 *swap*(3, 4)。这里的 3 和 4 被称为实际参数（简称实参）。

第 2 步，把实参赋值给函数声明中的 *a* 和 *b*。注意，这里的 *a* 和 *b* 与调用时的 *a* 和 *b* 是完全不同的。前面已经说过，实参最后将算出具体的值，*swap* 函数知道调用它的参数是 3 和 4，却不知道是怎么算出来的。函数声明中的 *a* 和 *b* 称为形式参数（简称形参）。

稍等一下，这里有个问题！这样一来，程序里有两个变量 *a*，一个在 *main* 函数里定义，一个是 *swap* 的形参，二者不会混淆吗？不会。函数（包括 *main* 函数）的形参和在该函数里定义的变量都被称为该函数的局部变量（*local variable*）。不同函数的局部变量相互独立，即无法访问其他函数的局部变量。需要注意的是，局部变量的存储空间是临时分配的，函数执行完毕时，局部变量的空间将被释放，其中的值无法保留到下次使用。与此对应的是全局变量（*global variable*）：此变量在函数外声明，可以在任何时候，由任何函数访问。需要注意的是，应该谨慎使用全局变量。

提示 4-11：函数的形参和在函数内声明的变量都是该函数的局部变量。无法访问其他函数的局部变量。局部变量的存储空间是临时分配的，函数执行完毕时，局部变量的空间将被释放，其中的值无法保留到下次使用。在函数外声明的变量是全局变量，可以被任何函数使用。操作全局变量有风险，应谨慎使用。

这样一来，函数的调用过程就可以简单理解成计算实参的值，赋值给对应的形参，然后把“当前代码行”转移到函数的首部。换句话说，在 *swap* 函数刚开始执行时，局部变量 *a* = 3, *b* = 4，二者的值是在函数调用时，由实参复制而来。

那么执行完毕后，函数又做了些什么呢？把返回值返回给调用它的函数，然后再次修改“当前代码行”，恢复到调用它的地方继续执行。等一下！函数是如何知道该返回到哪里继续执行的呢？为了解释这一问题，下面需要暂时把讨论变得学术一些——不要紧张，很快就会结束。

4.2.2 调用栈

还记得在讲解 *for* 循环时，笔者是如何建议的吗？多演示程序执行的过程，把注意力集中在“当前代码行”的转移和变量值的变化。这个建议同样适用于对函数的学习，只是要

增加一项内容——调用栈（Call Stack）。

调用栈描述的是函数之间的调用关系。它由多个栈帧（Stack Frame）组成，每个栈帧对应着一个未运行完的函数。栈帧中保存了该函数的返回地址和局部变量，因而不仅能在执行完毕后找到正确的返回地址，还很自然地保证了不同函数间的局部变量互不相干——因为不同函数对应着不同的栈帧。

提示 4-12: C 语言用调用栈（Call Stack）来描述函数之间的调用关系。调用栈由栈帧（Stack Frame）组成，每个栈帧对应着一个未运行完的函数。在 gdb^①中可以用 backtrace（简称 bt）命令打印所有栈帧信息。若要用 p 命令打印一个非当前栈帧的局部变量，可以用 frame 命令选择另一个栈帧。

在继续学习之前，建议读者试着调试一下刚才几个程序，除了关心“当前代码行”和变量的变化之外，再看看调用栈的变化。强烈建议读者在执行完 swap 函数的主体但还没有返回 main 函数之前，先看一下 swap 和 main 函数所对应的栈帧中 a 和 b 的值。如果受条件限制，在阅读到这里时没有办法完成这个实验，下面给出了用 gdb 完成上述操作的命令和结果。

第 1 步：编译程序。

```
gcc swap.c -std=c99 -g
```

生成可执行程序 a.exe（在 Linux 下是 a.out）。编译选项-g 告诉编译器生成调试信息。编译选项-std=c99 告诉编译器按照 C99 标准编译代码。

第 2 步：运行 gdb。

```
gdb a.exe
```

这样，gdb 在运行时会自动装入刚才生成的可执行程序。

第 3 步：查看源码。

```
(gdb) l
1      #include<stdio.h>
2      void swap(int a, int b){
3          int t = a; a = b; b = t;
4      }
5
6      int main(){
7          int a = 3, b = 4;
8          swap(3, 4);
9          printf("%d %d\n", a, b);
10         return 0;
```

这里(gdb)是 gdb 的提示符，字母 l 是输入的命令，为 list（列出程序清单）的缩写。正如代码所示，swap 函数的最后一行是第 4 行，当执行到这一行时，swap 函数的主体已经结

^① gdb 是一个功能强大的源码级调试器，虽然是基于命令的文本界面，但运用熟练后非常方便。关于 gdb 更多的介绍请参见附录 A。

束,但函数还没有返回。

第4步:加断点并运行。

```
(gdb) b 4
Breakpoint 1 at 0x401308: file swap.c, line 4.
(gdb) r
Starting program: D:\a.exe

Breakpoint 1, swap (a=4, b=3) at swap.c:4
4      }
```

其中, `b` 命令把断点设在了第4行, `r` 命令运行程序, 之后碰到了断点并停止。

第5步: 查看调用栈。

```
(gdb) bt
#0 swap (a=4, b=3) at swap.c:4
#1 0x00401356 in main () at swap.c:8
(gdb) p a
$1 = 4
(gdb) p b
$2 = 3
(gdb) up
#1 0x00401356 in main () at swap.c:8
8      swap(3, 4);
(gdb) p a
$3 = 3
(gdb) p b
$4 = 4
```

这一步是关键。根据 `bt` 命令, 调用栈中包含两个栈帧: #0 和 #1, 其中 0 号是当前栈帧——`swap` 函数, 1 号是其“上一个”栈帧——`main` 函数。这里甚至能看到 `swap` 函数的返回地址 `0x00401356`, 尽管不明确其具体含义。

使用 `p` 命令可以打印变量值。首先查看当前栈帧中 `a` 和 `b` 的值, 分别等于 4 和 3——这正是用三变量法交换后的结果。接下来用 `up` 命令选择上一个栈帧, 再次使用 `p` 命令查看 `a` 和 `b` 的值, 这次却得到 3 和 4, 为 `main` 函数中的 `a` 和 `b`。前面讲过, 在函数调用时, `a`、`b` 只起到了“计算实参”的作用。但实参被赋值到形参之后, `main` 函数中的 `a` 和 `b` 也完成了它们的使命。`swap` 函数甚至无法知道 `main` 函数中也有着和形参同名的 `a` 和 `b` 变量, 当然也就无法对其进行修改。最后要用 `q` 命令退出 `gdb`。

用了这么多篇幅解释调用栈和栈帧, 是因为无数的经验告诉笔者: 理解它们对于今后的学习和编程是至关重要的, 特别是递归——初学者学习语言的最大障碍之一, 调用栈将有助于理解。

4.2.3 用指针作参数

在了解了刚才的 swap 函数不能奏效的原因后，应该如何编写 swap 函数呢？答案是用指针。

程序 4-6 用函数交换变量（正确）

```
#include<stdio.h>
void swap(int* a, int* b)
{
    int t = *a; *a = *b; *b = t;
}
```

```
int main()
{
    int a = 3, b = 4;
    swap(&a, &b);
    printf("%d %d\n", a, b);
    return 0;
}
```

怎么样，是不是觉得不太习惯，却又有点似曾相识呢？不太习惯的是 int 和 a 中间的乘号，而似曾相识的是 swap(&a, &b) 这种变量名前面加 “&” 的用法——到目前为止，唯一采取这种用法的是 scanf 系列函数，而只有它改变了实参的值！

变量名前面加 “&” 得到的是该变量的地址。什么是“地址”呢？

提示 4-13：C 语言的变量都是放在内存中的，而内存中的每个字节都有一个称为地址（address）的编号。每个变量都占有一定数目的字节（可用 sizeof 运算符获得），其中第一个字节的地址称为变量的地址。

下面用 gdb 来调试上面的程序，看看它和程序 4-5 有什么不同。前 4 步是一样的，可直接看调用栈。

```
(gdb) bt
#0 swap (a=0x22ff74, b=0x22ff70) at swap2.c:4
#1 0x0040135c in main() at swap2.c:8
(gdb) p a
$1 = (int *) 0x22ff74
(gdb) p b
$2 = (int *) 0x22ff70
(gdb) p *a
$3 = 4
(gdb) p *b
```

```
$4 = 3
(gdb) up
#1 0x0040135c in main() at swap2.c:8
8      swap(&a, &b);
(gdb) p a
$5 = 4
(gdb) p b
$6 = 3
(gdb) p &a
$7 = (int *) 0x22ff74
(gdb) p &b
$8 = (int *) 0x22ff70
```

在打印 `a` 和 `b` 的值时，得到了诡异的结果——`(int *) 0x22ff74` 和 `(int *) 0x22ff70`。数值 `0x22ff74` 和 `0x22ff70` 是两个地址（以 `0x` 开头的整数以十六进制表示，在这里暂时不需了解细节），而前面的 `(int *)` 表明 `a` 和 `b` 是指向 `int` 类型的指针。

提示 4-14：用 `int* a` 声明的变量 `a` 是指向 `int` 型变量的指针。赋值 `a = &b` 的含义是把变量 `b` 的地址存放在指针 `a` 中，表达式 `*a` 代表 `a` 指向的变量，既可以放在赋值符号的左边（左值），也可以放在右边（右值）。

注意：`*a` 是指“`a` 指向的变量”，而不仅是“`a` 指向的变量所拥有的值”。理解这一点相当重要。例如，`*a = *a + 1` 就是让 `a` 指向的变量自增 1。甚至可以把它写成 `(*a)++`。注意不要写成 `*a++`，因为“`++`”运算符的优先级高于“取内容”运算符“`*`”，实际上会被解释成 `*(a++)`。

有了指针，C 语言变得复杂了很多。一方面，需要了解更多底层的内容才能彻底解释一些问题，包括运行时的地址空间布局，以及操作系统的内存管理方式等。另一方面，指针的存在，使得 C 语言中变量的说明变得异常复杂——你能轻易地说出用 `char * const (*next)()` 声明的 `next` 是什么类型的吗^①？毫不夸张地说，指针是程序员（不仅是初学者）杀手。

既然如此，那应当如何使用指针呢？别忘了本书的背景——算法竞赛。算法竞赛的核心是算法，没有必要纠缠如此复杂的语言特性。了解底层的细节是有益的（事实上，前面已经介绍了一些底层细节），但在编程时应尽量避免，只遵守一些注意事项即可。

提示 4-15：千万不要滥用指针，这不仅会把自己搞糊涂，还会让程序产生各种奇怪的错误。事实上，本书的程序会很少使用指针。

再次回到对正确 `swap` 程序的调试。在 `swap` 程序中，`a` 和 `b` 都是局部变量，在函数执行完毕以后就不复存在了，但是 `a` 和 `b` 里保存的地址却依然有效——它们是 `main` 函数中的局部变量 `a` 和 `b` 的地址。在 `main` 函数执行完毕之前，这两个地址将始终有效，并且分别指向 `main` 函数的局部变量 `a` 和 `b`。程序交换的是 `*a` 和 `*b`，也就是 `main` 函数中的局部变量 `a` 和 `b`。

^① 这是一个指向函数的指针，该函数返回一个指针，该指针指向一个只读的指针，此指针指向一个字符变量。

4.2.4 初学者易犯的错误

这个 `swap` 函数看似简单，但初学者还是很容易写错。一种典型的错误写法是：

```
void swap(int* a, int* b)
{
    int *t = a; a = b; b = t;
}
```

此写法交换了 `swap` 函数的局部变量 `a` 和 `b`（辅助变量 `t` 必须是指针。`int t = a` 是错误的），但却始终没有修改它们指向的内容，因此 `main` 函数中的 `a` 和 `b` 不会改变。另一种错误写法是：

```
void swap(int* a, int* b)
{
    int *t;
    *t = *a; *a = *b; *b = *t;
}
```

这个程序错在哪里？`t` 是一个指向 `int` 型的指针，因此 `*t` 是一个整数。用一个整数作为辅助变量去交换两个整数有何不妥？事实上，如果用这个函数去替换程序 4-6，很可能会得到“43”的正确结果。为什么笔者要坚持说它是错误的呢？

问题在于，`t` 存储的地址是什么？也就是说 `t` 指向哪里？因为 `t` 是一个变量（指针也是一个变量，只不过类型是“指针”），所以根据规则，它在赋值之前是不确定的。如果这个“不确定的值”所代表的内存单元恰好是能写入的，那么这段程序将正常工作；但如果它是只读的，程序可能会崩溃。读者可尝试赋初值 `int *t = 0`，看看内存地址“0”能不能写。

至此，终于初步理解了地址和指针。尽管只是初步理解，但是为将来的学习奠定了良好的基础。指针有很多巧妙但又令人困惑的用法。如果有一种语法，但在完整地学习了本书后始终没有看到此语法被使用，那么这通常意味着这个语法不必学（至少在算法竞赛中不必用到）。事实上，笔者在编写本书的例程时，首先考虑的是要通俗易懂，避开复杂的语言特性，其次才是简洁和效率。

4.2.5 数组作为参数和返回值

如何把数组作为参数传递给函数？先来看下面的例子。

程序 4-7 计算数组的元素和（错误）

```
int sum(int a[]) {
    int ans = 0;
    for(int i = 0; i < sizeof(a); i++)
        ans += a[i];
    return ans;
}
```


这个函数是错误的，因为 `sizeof(a)` 无法得到数组的大小。为什么会这样？因为把数组作为参数传递给函数时，实际上只有数组的首地址作为指针传递给了函数。换句话说，在函数定义中的 `int a[]` 等价于 `int *a`。在只有地址信息的情况下，是无法知道数组里有多少个元素的。

正确的做法是加一个参数，即数组的元素个数。

程序 4-8 计算数组的元素和（正确）

```
int sum(int* a, int n) {
    int ans = 0;
    for(int i = 0; i < n; i++)
        ans += a[i];
    return ans;
}
```

在上面的代码中，直接把参数 `a` 写成了 `int* a`，暗示 `a` 实际上是一个地址。在函数调用时 `a` 不一定非要传递一个数组，例如：

```
int main() {
    int a[] = {1, 2, 3, 4};
    printf("%d\n", sum(a+1, 3));
    return 0;
}
```

提示 4-16：以数组为参数调用函数时，实际上只有数组首地址传递给了函数，需要另加一个参数表示元素个数。除了把数组首地址本身作为实参外，还可以利用指针加减法把其他元素的首地址传递给函数。

指针 `a+1` 指向 `a[1]`，即 2 这个元素（数组元素从 0 开始编号）。因此函数 `sum` “看到” `{2, 3, 4}` 这个数组，因此返回 9。一般地，若 `p` 是指针，`k` 是正整数，则 `p+k` 就是指针 `p` 后面第 `k` 个元素，`p-k` 是 `p` 前面的第 `k` 个元素，而如果 `p1` 和 `p2` 是类型相同的指针，则 `p2-p1` 是从 `p1` 到 `p2` 的元素个数（不含 `p2`）。下面是 `sum` 函数的另外两种写法。

程序 4-9 计算左闭右开区间内的元素和（两种写法）

写法一：

```
int sum(int* begin, int* end) {
    int n = end - begin;
    int ans = 0;
    for(int i = 0; i < n; i++)
        ans += begin[i];
    return ans;
}
```

写法二：

```
int sum(int* begin, int* end) {
    int *p = begin;
```



```

int ans = 0;
for(int *p = begin; p != end; p++)
    ans += *p;
return ans;
}

```

其中写法一先进行了一次指针减法，算出了从 `begin` 到 `end`（不含 `end`）的元素个数 `n`，然后再像前面那样把 `begin` 作为“数组名”进行累加。写法二看起来更“高级”，事实上也更具一般性，用一个新指针 `p` 作为循环变量，同时累加其指向的值。这两个函数的调用方式与之前相似，例如，声明了一个长度为 10 的数组 `a`，则它的元素之和就是 `sum(a, a+10)`；若要计算 `a[i], a[i+1], ..., a[j]`，则需要调用 `sum(a+i, a+j+1)`。

`sum` 的最后两种写法及其调用方式非常重要（将在第 5 章中继续讨论），请读者仔细体会。

把数组作为指针传递给函数时，数组内容是可以修改的。因此如果要写一个“返回数组”的函数，可以加一个数组参数，然后在函数内修改这个数组的内容。不过在算法竞赛中经常采取其他做法，原因在第 5 章会做进一步的说明。

4.2.6 把函数作为函数的参数

把函数作为函数的参数？看上去挺奇怪的，但实际上有一个非常典型的应用——排序。

例题 4-1 古老的密码 (Ancient Cipher, NEERC 2004, UVa1339)

给定两个长度相同且不超过 100 的字符串，判断是否能把其中一个字符串的各个字母重排，然后对 26 个字母做一个一一映射，使得两个字符串相同。例如，JWPUDJSTVP 重排后可以得到 WJDUPSJPVT，然后把每个字母映射到它前一个字母（B→A, C→B, ..., Z→Y, A→Z），得到 VICTORIOUS。输入两个字符串，输出 YES 或者 NO。

【分析】

既然字母可以重排，则每个字母的位置并不重要，重要的是每个字母出现的次数。这样可以先统计出两个字符串中各个字母出现的次数，得到两个数组 `cnt1[26]` 和 `cnt2[26]`。下一步需要一点想象力：只要两个数组排序之后的结果相同，输入的两个串就可以通过重排和一一映射变得相同。这样，问题的核心就是排序。

C 语言的 `stdlib.h` 中有一个叫 `qsort` 的库函数，实现了著名的快速排序算法。它的声明是这样的：

```

void qsort ( void *base, size_t num, size_t size, int (* comparator ) ( const
void *, const void * ) );

```

前 3 个参数不难理解，分别是待排序的数组起始地址、元素个数和每个元素的大小。最后一个参数比较特别，是一个指向函数的指针，该函数应当具有这样的形式：

```

int cmp(const void *, const void *) { ... }

```

这里的新内容是指向常数的“万能”的指针：`const void *`，它可以通过强制类型转化变成任意类型的指针。对于本题来说，排序的对象是整型数组，因此要这样写：